

Análisis arquitectónico: microservicios y API Gateway en entornos de alta demanda

Docente: Ing. Guerrero Ulloa Gleiston, PhD.

Estudiante: Cando Moreno Robinson

Tema: FORM-IC-01 — Investigación Individual — Arquitecturas Distribuidas en la Industria

Descripción del Caso

Las arquitecturas de microservicios organizan las aplicaciones distribuidas como un conjunto de servicios independientes para lograr escalabilidad y mantenibilidad. En este modelo, el API Gateway actúa como un punto de entrada único que centraliza las solicitudes, gestionando autenticación, enrutamiento dinámico y balanceo de carga. Líderes como Netflix y Amazon son unos de los que utilizan este patrón para desplegar cambios rápidamente sin afectar toda la aplicación [1].

Justificación del Estilo Arquitectónico

Las organizaciones adoptan este estilo porque permite desplegar servicios individualmente, escalando solo los módulos con mayor demanda. El desacoplamiento tecnológico facilita que los equipos trabajen independientemente con diferentes stacks. Además, la naturaleza distribuida mejora significativamente la tolerancia a fallos un error en un servicio secundario no detiene el núcleo del sistema [1].

Beneficios Obtenidos

La principal ventaja es la adaptación. Al aislar funcionalidades, el sistema tolera interrupciones en servicios o bases de datos sin caída total. El API Gateway centraliza seguridad (SSL/TLS, autenticación), reduciendo la duplicación de código y asegurando comportamiento consistente. El despliegue independiente de servicios acelera los ciclos de desarrollo y mejora la experiencia del usuario.

Desafíos Encontrados

La descentralización introduce complejidad operativa. Investigaciones recientes han catalogado más de 117 tipos de fallos potenciales en microservicios relacionados con red, seguridad y rendimiento [1]. La naturaleza asíncrona dificulta testing y trazabilidad de errores, exigiendo herramientas avanzadas para el monitoreo. Las pruebas mediante inyección de fallos son críticas para validar que servicios clave toleren fallos en dependencias externas [2].

Aplicación al Proyecto: Sistema de Soporte Técnico

Se propone segmentar la aplicación en microservicios independientes: Gestión de Tickets, Notificaciones y Módulo de IA. Esto evitaría que el procesamiento pesado de la IA bloquee

la creación de tickets. Un API Gateway centralizaría entrada de los clientes y técnicos, asegurando que solo peticiones autenticadas lleguen a servicios internos. Se estarían aplicando pruebas con inyección de fallos [2] para validar que, si la base de datos de notificaciones falla, el servicio maneje la excepción correctamente sin interrumpir al cliente, usando colas de eventos y reintentos automáticos.

Referencias

- [1] F. Silva, V. Lelli, I. Santos, and R. Andrade, “Towards a Fault Taxonomy for Microservices-Based Applications,” *ACM International Conference Proceeding Series*, pp. 247–256, Oct. 2022. doi: <https://doi.org/10.1145/3555228.3555245>
- [2] M. Assad, C. S. Meiklejohn, H. Miller, and S. Krusche, “Can My Microservice Tolerate an Unreliable Database? Resilience Testing with Fault Injection and Visualization,” *Proceedings - International Conference on Software Engineering*, pp. 54–58, May 2024. doi: <https://doi.org/10.1145/3639478.3640021>